

## Chapter 5. Backing Up and Restoring etcd

Kubernetes stores both the declared and observed states of the cluster in the distributed etcd key-value store. It's important to have a backup plan in place that can help you with restoring the data in case of data corruption. Backing up the data should happen periodically in short time frames to avoid losing as much historical data as possible.

---

### COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Manage the lifecycle of Kubernetes clusters
- 

## Using the etcd Administration Utilities

The backup process stores the etcd data in a *snapshot file*. This snapshot file can be used to restore the etcd data at any given time. You can encrypt the snapshot file to protect sensitive information. The tool `etcdctl` is used to create a backup snapshot file. Restoring the etcd data from the snapshot file requires the use of the tool `etcdutl`.

As an administrator, you will need to understand how to use the tools for both operations. You may need to install `etcdctl` and `etcdutl` if they are not available on the control plane node yet. You can find [installation instructions](#) in the etcd GitHub repository. [Figure 5-1](#) visualizes the etcd backup and restoration process.

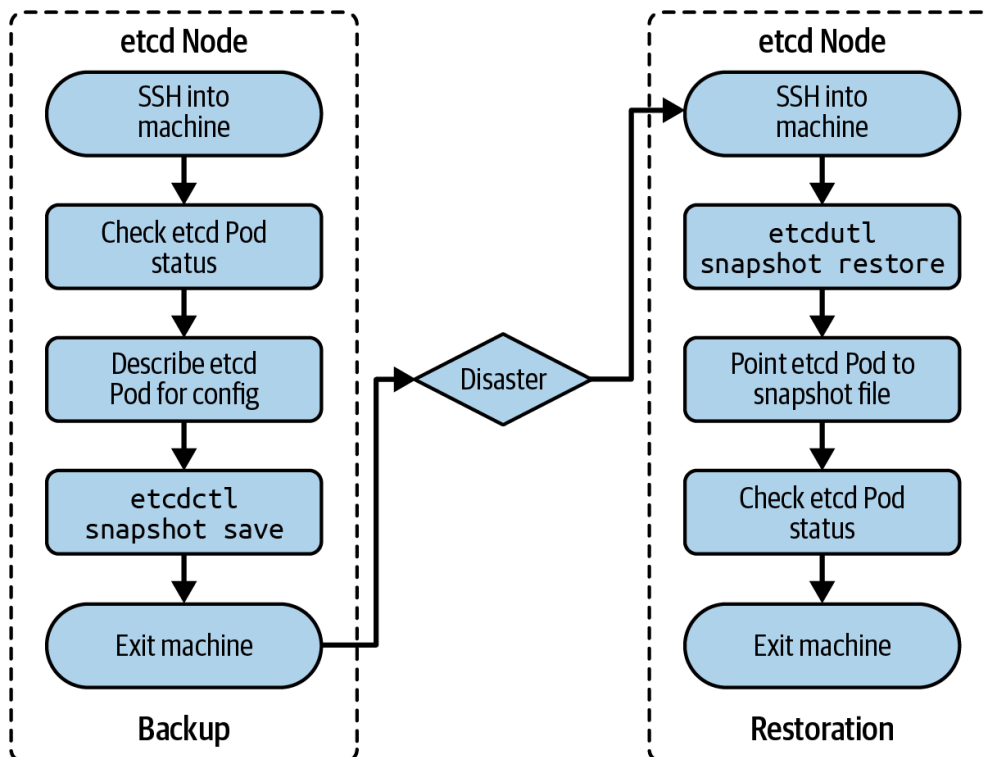


Figure 5-1. Process for backing up and restoring etcd

Depending on your cluster topology, your cluster may consist of one or many etcd instances. Refer to [“Managing a Highly Available Cluster”](#) for more information on how to set it up. The following sections explain a single-node etcd cluster setup. You can find [additional instructions](#) on the backup and restoration process for multinode etcd clusters in the official Kubernetes documentation.

## Backing Up etcd

Open an interactive shell to the machine hosting etcd using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.04 LTS:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-51-generic x86_64)
...
```

Check the installed version of `etcdctl` to verify that the tool has been installed. On this node, the version is 3.5.15:

```
$ etcdctl version
etcdctl version: 3.5.15
API version: 3.5
```

Etcd is deployed as a Pod in the `kube-system` namespace. Inspect the version by describing the Pod. In the following output, you will find that

the version is 3.5.15-0:

```
$ kubectl get pods -n kube-system
```

| NAME                    | READY | STATUS  | RESTARTS | AGE |
|-------------------------|-------|---------|----------|-----|
| ...                     |       |         |          |     |
| etcd-kube-control-plane | 1/1   | Running | 0        | 3d  |
| ...                     |       |         |          |     |

```
$ kubectl describe pod etcd-kube-control-plane -n kube-system
```

```
...
```

```
Containers:
```

```
  etcd:
```

```
    Container ID:  containerd://47a6cf3ed27d455be6c9b782d2e35ee77b429ee5c03c6c3d6282628f6492b15
```

```
    Image:         registry.k8s.io/etcd:3.5.15-0
```

```
    Image ID:      registry.k8s.io/etcd@sha256:a6dc63e6e8cfa0307d7851762fab629afb18f28d8aa3fab5a6e91b4af60026a
```

```
...
```

The same `describe` command reveals the configuration of the etcd service. Look for the value of the option `--listen-client-urls` for the endpoint URL. In the following output, the host is `localhost`, and the port is `2379`. The server certificate is located at `/etc/kubernetes/pki/etcd/server.crt` defined by the option `--cert-file`. The CA certificate can be found at `/etc/kubernetes/pki/etcd/ca.crt` specified by the option `--trusted-ca-file`:

```
$ kubectl describe pod etcd-kube-control-plane -n kube-system
```

```
...
```

```
Containers:
```

```
  etcd:
```

```
    ...
```

```
    Command:
```

```
      etcd
```

```
    ...
```

```
      --cert-file=/etc/kubernetes/pki/etcd/server.crt
```

```
      --key-file=/etc/kubernetes/pki/etcd/server.key
```

```
      --listen-client-urls=/etc/kubernetes/pki/etcd/server.key
```

```
      --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

```
...
```

Use the `etcdctl` command to create the backup with version 3 of the tool. For a good starting point, copy the command from the [official Kubernetes documentation](#). Provide the mandatory command-line options `--cacert`, `--cert`, and `--key`. The option `--endpoints` is not needed as we are running the command on the same server as etcd. After running the command, the file `/opt/etcd-backup.db` has been created:

```
$ sudo ETCDCTL_API=3 etcdctl --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/etcd-backup.db
...
Snapshot saved at /opt/etcd-backup.db
```

Exit the node using the `exit` command:

```
$ exit
logout
...
```

## Restoring etcd

You created a backup of etcd and stored it in a safe space. There's nothing else to do at this time. Effectively, it's your insurance policy that becomes relevant when disaster strikes. In the case of a disaster scenario—say, the data in etcd gets corrupted or the machine managing etcd experiences a physical storage failure—that's the time when you want to pull out the etcd backup for restoration.

To restore etcd from the backup, use the `etcdctl snapshot restore` command. At a minimum, provide the `--data-dir` command-line option.

---

### DEPRECATED ETCDCTL COMMAND

The `etcdctl snapshot restore` command has been deprecated and will be [removed](#) with the release of etcd 3.6. Use the `etcdctl snapshot restore executable` instead.

---

Open an interactive shell to the machine hosting etcd using the `ssh` command:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-51-generic x86_64)
...
```

After running the following command, you should be able to find the restored backup in the directory `/var/lib/from-backup`:

```
$ sudo ETCDCTL_API=3 etcdctl --data-dir=/var/lib/etcd-from-backup snapshot restore
/opt/etcd-backup.db
...
$ sudo ls /var/lib/etcd-from-backup
member
```

Edit the YAML manifest of the etcd Pod, which can be found at `/etc/kubernetes/manifests/etcd.yaml`. Change the value of the attribute `spec.volumes.hostPath` with the name `etcd-data` from the original value `/var/lib/etcd` to `/var/lib/etcd-from-backup`:

```
$ cd /etc/kubernetes/manifests/
$ sudo vim etcd.yaml
...
spec:
  volumes:
    ...
  - hostPath:
      path: /var/lib/etcd-from-backup
      type: DirectoryOrCreate
      name: etcd-data
...
```

The `etcd-kube-control-plane` Pod will be re-created, and it points to the restored backup directory:

```
$ kubectl get pod etcd-kube-control-plane -n kube-system
```

| NAME                    | READY | STATUS  | RESTARTS | AGE  |
|-------------------------|-------|---------|----------|------|
| etcd-kube-control-plane | 1/1   | Running | 0        | 5m1s |

In case the Pod doesn't transition into the `Running` status, try to delete it manually with the command `kubectl delete pod etcd-kube-control-plane -n kube-system`.

Exit the node using the `exit` command:

```
$ exit
logout
...
```

## Summary

Backing up the etcd database should be performed as a periodic process to prevent the loss of crucial data in the event of node or storage corrup-

tion. You can use the tool `etcdctl` to back up and the tool `etcdutl` to restore etcd from the control plane node or via an API endpoint.

## Exam Essentials

### *Practice backing up etcd*

Backing up etcd requires the installation of a compatible version of the `etcdctl` executable. You can identify the version of etcd by inspecting the container image tag used to run the etcd Pod (assuming we are talking about a cluster without [high-availability characteristics](#)). The etcd process operates within a container inside the Pod, and examining the Pod's configuration reveals the command-line flags required to perform the backup operation. In the exam, you can assume that the `etcdctl` executable has been preinstalled.

### *Know how to restore etcd*

Restoring etcd requires the use of the executable `etcdutl`. You will need to point the command to the snapshot file created in the backup process, and to a target directory used to extract the etcd data into. Just extracting the etcd data into a directory doesn't tell the etcd process to use it. You need to configure the host path to the directory in the configuration for etcd.

## Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Navigate to the directory `app-a/ch05/etcd-backup-restore` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Start up the VMs running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes.

SSH into the control plane node host machine. Identify the Pod that runs the etcd executable. Inspect the details of the Pod to find out which version of etcd it is running. Write the version to the file `etcd-version.txt`.

Once done, shut down the cluster using `vagrant destroy -f`.

*Prerequisite:* This exercise requires the installation of the tools [Vagrant](#) and a [VMware provider](#).

2. Navigate to the directory `app-a/ch05/etcd-backup-restore` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Start up the VMs running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes.

The `etcdctl` and `etcdutl` tool have been preinstalled on the node `kube-control-plane`. Back up etcd to the snapshot file `/opt/etcd.bak`. Restore etcd from the snapshot file. Use the data directory `/var/bak`.

Once done, shut down the cluster using `vagrant destroy -f`.

*Prerequisite:* This exercise requires the installation of the tools [Vagrant](#) and a [VMware provider](#).